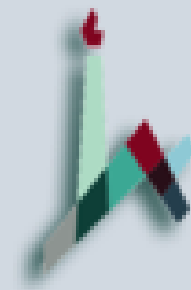


AI Agents & Crews

DR. ZVI BEN AMI



AI Agents & Crews

AGENDA

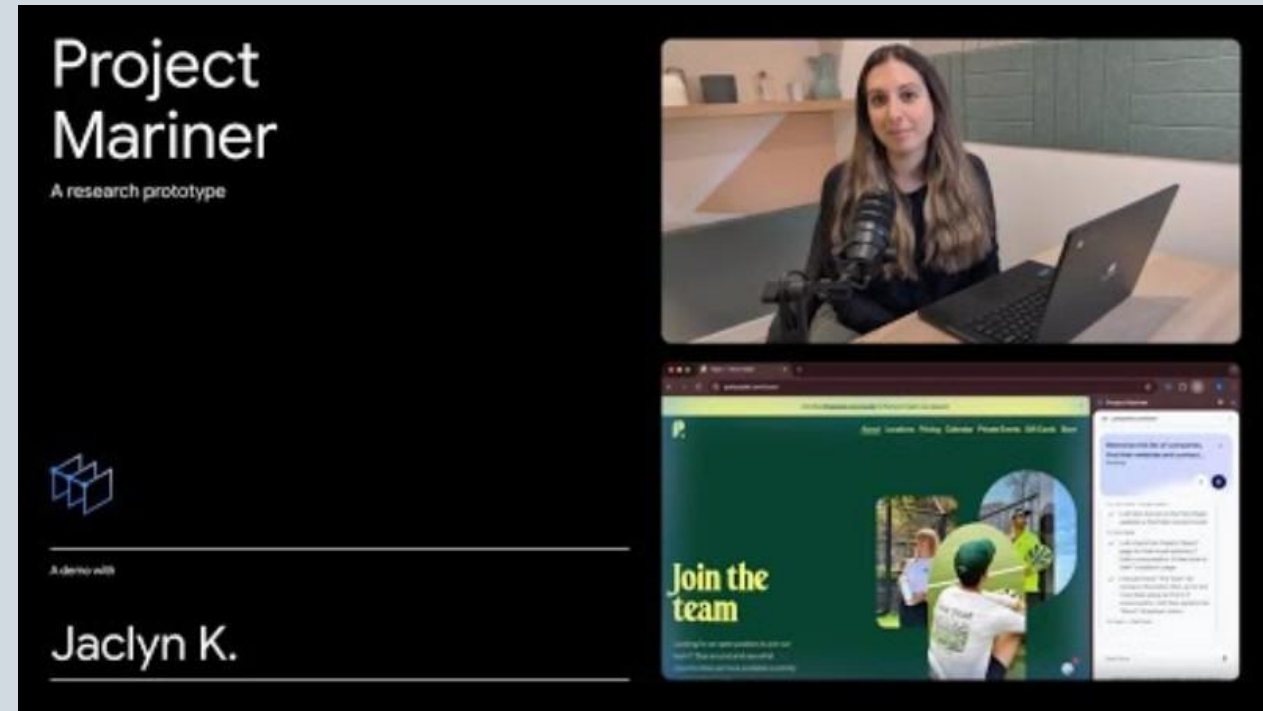
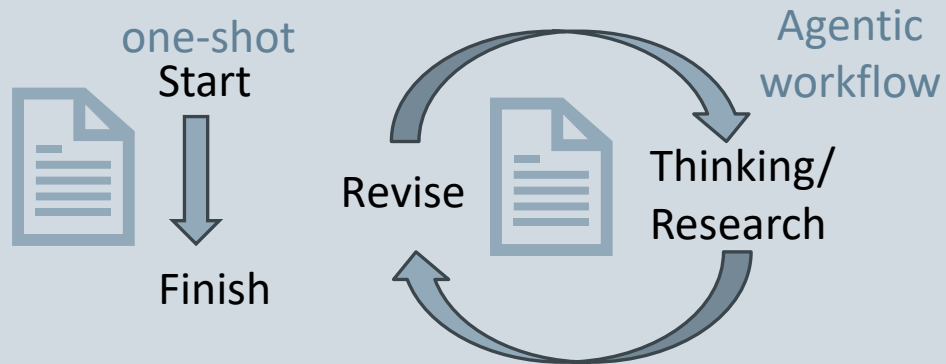
- Motivation: Why Agents?
- Agent Types & Capabilities
- Frameworks & Memory Models
- Workflow Composition & Planning
- Multi-Agent Orchestration
- Security & Deployment Best Practices
- Summary, Q&A, Resources

LEARNING OBJECTIVES

- By the end of this session, you will be able to:
 - Explain the purpose and types of AI agents
 - Build tool-using and planning agents using LangChain, AutoGen, or CrewAI
 - Orchestrate multi-agent workflows using roles and memory

AI Agents

- Pipelines are rigid and require predefined steps.
- AI agents can autonomously plan, reason, and act.
- Enable goal-directed behavior, external tool use, and memory.
- Agentic workflow vs. one-shot prompt
 - Break down an overall task into different steps.
 - E.g., Write an essay:



AI Agent use case



Hey I am John, I work at Micro-Cool Corporation. I am planning a 2 Skiing vacation from mid of December if I have leaves. I Need help with vacation planning.

Collection of Tasks & Decisions

- ☐ Check my Leave balance
- ☐ Research locations for skiing
- ☐ Check weather report for location
- ☐ Book flight
- ☐ Book hotel

Limitations of LLM's to handle complex user requests

User request

I am planning a 2 Skiing vacation from mid of December if I have leaves. I Need help with vacation planning.



Check my Leave balance



Research locations for skiing



Check weather report for location



Book flight



Book hotel

Limitations and Challenges of LLM based Applications

- Answer generic questions based on publicly available training data **but not company specific data**
- No way to **access current data or external API's** – Today's weather, book hotel etc.
- Answer only questions based **on training data with cut off date**

Limitations of LLM's to handle complex user requests

User request

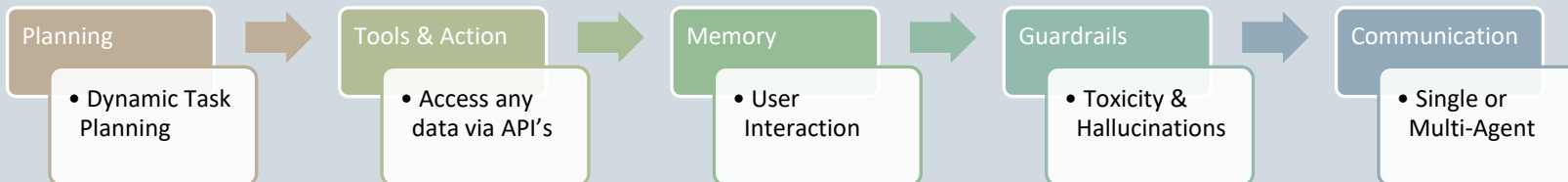
I am planning a 2 Skiing vacation from mid of December if I have leaves. I Need help with vacation planning.



Agentic App



1. Task Decomposition & Planning
 - ✓ Check my Leave balance
 - ✓ Research locations for skiing
 - ✓ Check weather report for location
 - ✓ Book flight
 - ✓ Book hotel
2. Tools & Actions
 - ✓ Retrieve HR Data using RAG
 - ✓ Search the Web for skiing location
 - ✓ Call Weather API
 - ✓ Call Hotel.com API
3. Memory for end user interaction
4. Guardrails – Toxicity, Hallucinations
5. Agent Communication



Agent Types Overview

- Reactive:
 - Stateless, rule-based
 - e.g., FAQ bots
- Tool-Using:
 - Uses APIs or external tools
 - e.g., calculator, search
- Planning:
 - Decomposes goals into sub-steps
 - e.g., trip planner, report writer

Agentic AI

- Uses **sophisticated reasoning and iterative planning** to **autonomously** solve complex, multi-step problems.

Agentic AI



Intelligent
Autonomous Systems



Plan, Reason and Act

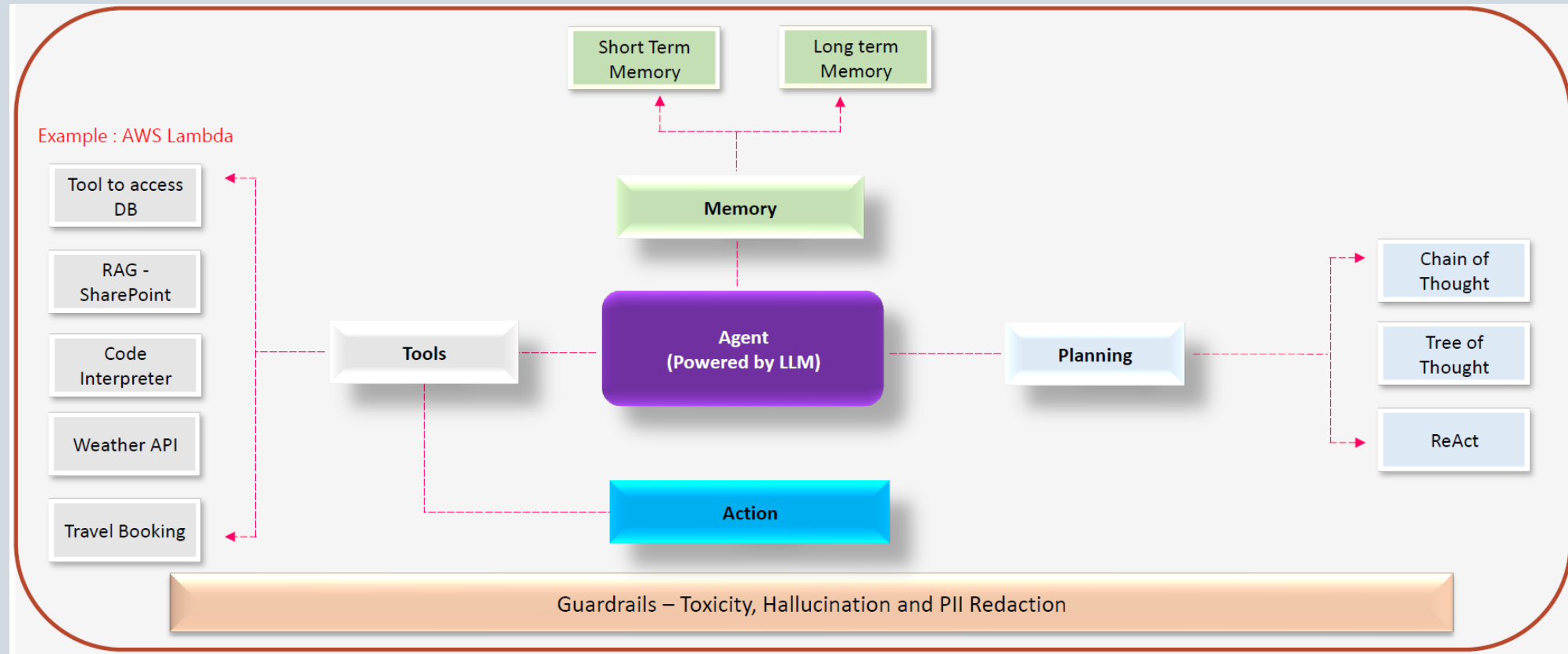


Are able to access
enterprise data



Are able to use Tools

AI Agent Architecture



Agent Frameworks

- What are Agent Frameworks?
 - Libraries or platforms that provide pre-built components and abstractions for developing AI agents.
 - Simplify the creation of agents by offering tools for memory management, tool integration, planning, and multi-agent communication.
 - Allow developers to focus on agent logic rather than low-level implementation details.
- Key Benefits:
 - Accelerated development.
 - Standardized approaches for common agent functionalities.
 - Easier integration with different models and tools.
 - Support for various agent architectures (e.g., ReAct, multi-agent systems).

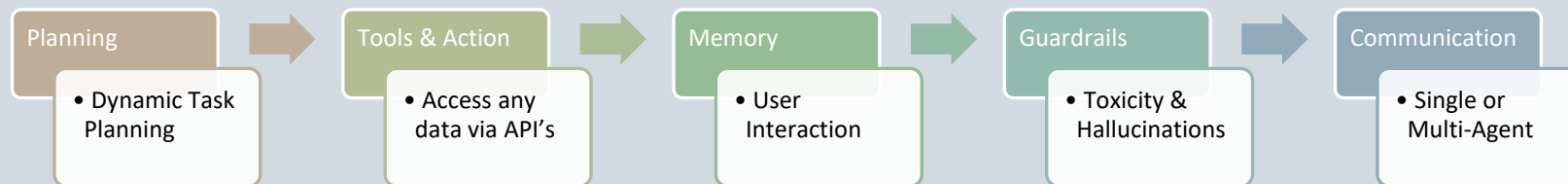
1. Planning

- **Planning without Feedback**

- Chain-of-Thought (CoT)
 - Enables complex reasoning capabilities for the LLM through intermediate reasoning steps
- Tree of Through Reasoning (ToT)
 - Decomposes the problem into multiple thought steps, generates multiple thoughts per step, creating a tree structure

- **Planning with Feedback**

- ReAct
 - Advanced **prompting technique** that combines **Reasoning** and **Action** within a prompt.
- Reflexion
 - A framework to reinforce language-based agents through linguistic feedback.



2. Tools Calling

User request

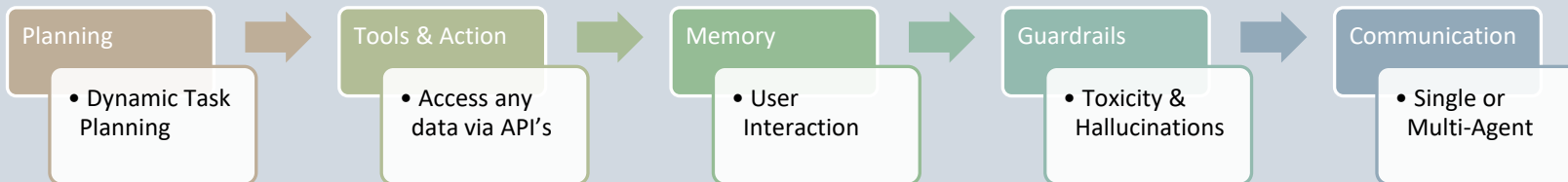
I am planning a 2 Skiing vacation from mid of December if I have leaves. I Need help with vacation planning.



Agentic App

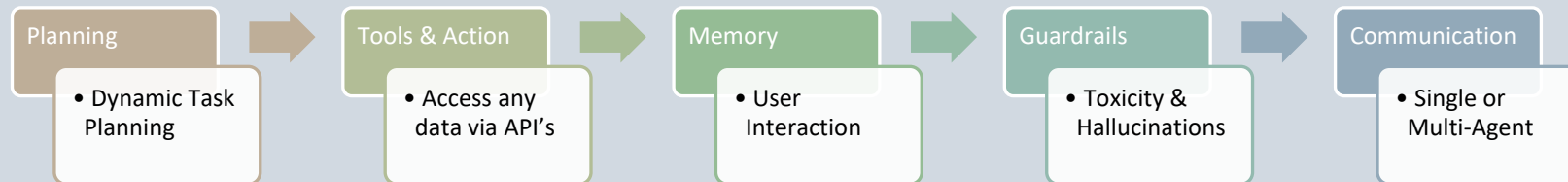


- ❖ Tool 1 – Call Weather API
 - Execute: Function written in AWS Lambda
- ❖ Tool 1 – Call Hotel Booking API
 - Execute: Function written in AWS Lambda
- ❖ Tool 3 – Code Execution
 - Execute: Ability to execute code generated by Agent (LLM)
- ❖ Tool 4 – Call to HR system using RAG
 - Execute: RAG app - Bedrock Knowledge Bases



3. Agentic Memory

- Remember – LLM's are Stateless
 - (No memory)
- Short term memory
 - Agent retains the conversation and user input during the particular session
- Long term memory
 - Allows agent to retain conversation across all previous interactions



Memory in Action

- **Short-Term Memory (Session-based)**

- Example: Medical triage bot
- *User*: “I have stomach pain.”
- *Bot*: Remembers context → later when user says “It started yesterday,” the bot links it back to “stomach pain” instead of asking again.

- **Long-Term Memory (Persistent across sessions)**

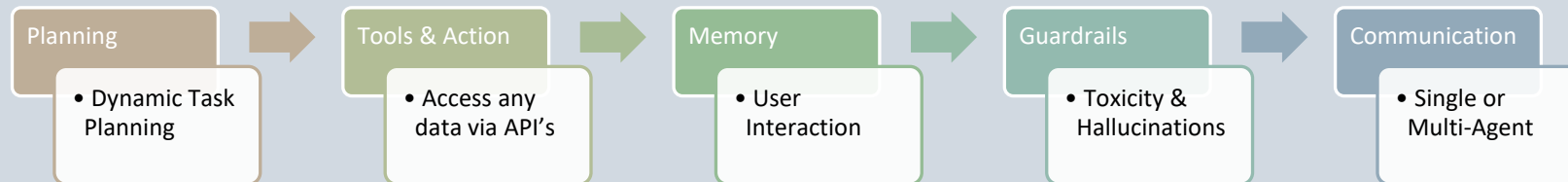
- Example: Personal tutor
- Session 1: Learner struggles with linear algebra.
- Session 2: Tutor recalls → *“Last time we reviewed eigenvalues. Shall we continue with eigenvectors today?”*

- **Hybrid Memory (Context + DB)**

- Example: Travel planner
- Session 1: Books flights for Paris.
- Session 2 (weeks later): *“Add museum tickets.”*
- Agent retrieves past itinerary from vector DB and updates plan seamlessly.

4. Guardrails

- Guardrails allow you to build responsible AI applications
 - Domain Relevance - Answer questions relevant to domain of Agent (Insurance, Banking etc.)
 - Toxicity Control - Identify and flag content that may be harmful, offensive, or inappropriate.
 - Hallucination Prevention - Hallucinations refer to the phenomenon where the LLM generates an output that is plausible but factually incorrect or made-up



Guardrails in Action

- **Scenario 1 – Toxicity Control**

- A customer support agent receives: *“Your service is garbage, you should all be fired.”*
- **Without guardrails:** Agent echoes or amplifies insult.
- **With guardrails:** Agent de-escalates → *“I understand your frustration. Let me help resolve this issue.”*

- **Scenario 2 – Domain Relevance**

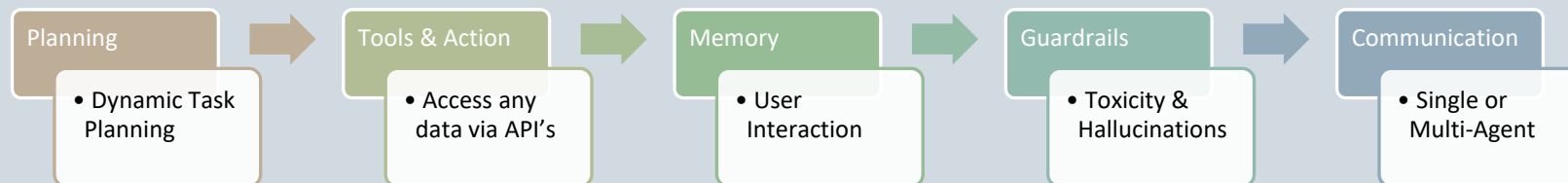
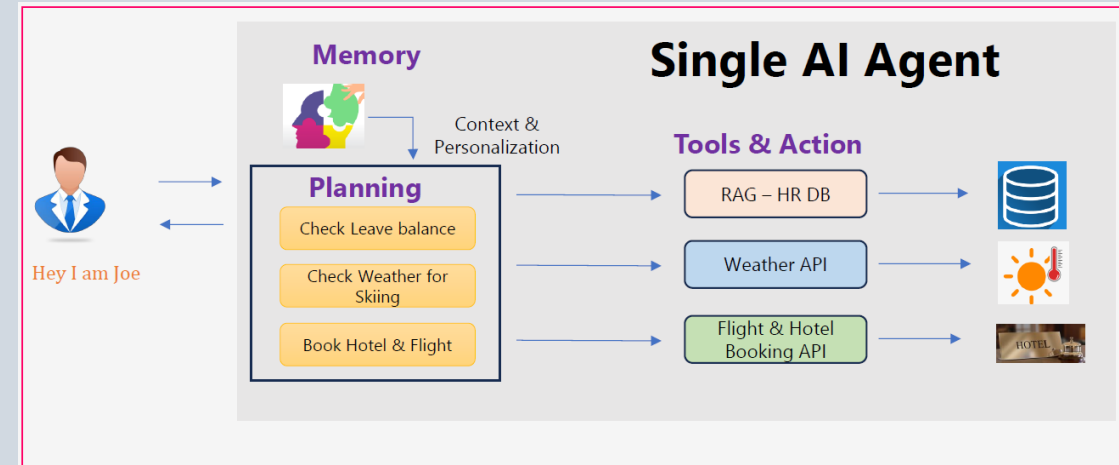
- Banking agent asked: *“Write me a poem about cats.”*
- **Without guardrails:** Outputs a cat poem.
- **With guardrails:** Responds → *“I can only answer finance-related questions. Would you like help with credit cards or savings?”*

- **Scenario 3 – Hallucination Prevention**

- User: *“What’s my account balance?”*
- **Without guardrails:** LLM fabricates a number.
- **With guardrails:** Routes to secure API, responds only with verified data.

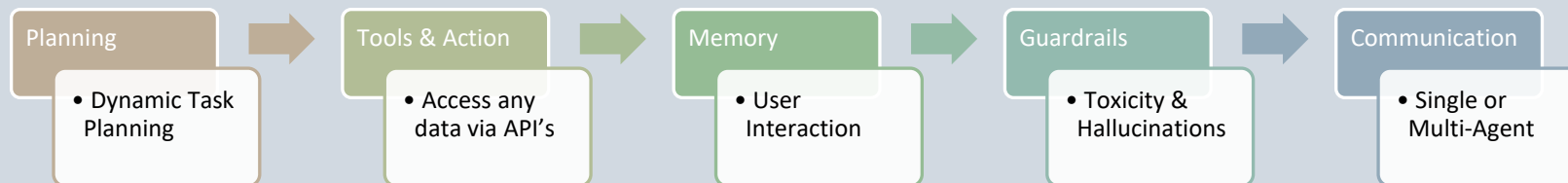
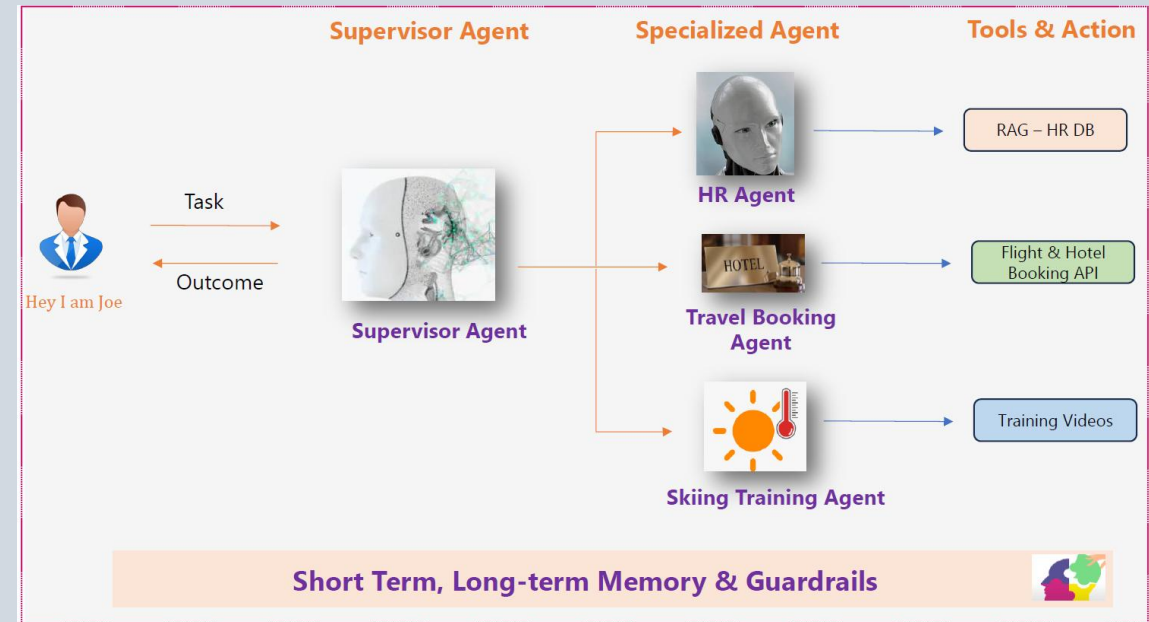
4. Agent Communication – Single Agent

- **Single Agent**
 - Single Agent powered by LLM.
 - Does user task decomposition.
 - Routes to various Tools at disposal.
 - Good for simple Use Cases.
- **Challenges**
 - Single Agent can get confused once complexity increases.
 - Less modular.
 - Changes in one service can impact multiple areas (Think Micro-Services).



4. Agent Communication – Single Agent

- **Supervisor Agent**
 - for Routing between Agents
- **Specialized Agents**
 - with specific tasks
- **Tools Calling**
 - for each Agent
- **Inter-Agent Communication**
 - Memory Sharing
 - Guardrails



Agent Frameworks

- LangChain:
 - Tool use, ReAct, Planner.
- AutoGen:
 - Multi-agent chat loops.
- CrewAI:
 - Role-based collaboration.

Feature	LangChain	AutoGen	CrewAI
Tool use	✓	✓	✓
Multi-agent support	✗	✓ (chat loops)	✓ (roles/tasks)
Planning	✓	✗	✓ (via task chains)
Deployment maturity	High	Experimental	Moderate

Manus.im: Real-World Implementation of Advanced Agent Architecture

- What is Manus?
 - General AI agent that turns thoughts into actions
 - Developed by Singapore-based startup Monica
 - Autonomous system for complex online tasks without continuous human guidance
 - Invitation-only platform at <https://manus.im/>
- Multi-Agent Architecture:
 - Uses multiple AI models (Claude 3.5 Sonnet, Alibaba Qwen)
 - Various independently operating specialized agents
 - Dynamic task decomposition algorithm
 - Coordinating orchestration layer for sub-agents:
 - Planning agents
 - Information retrieval agents
 - Code generation agents
 - Tool execution agents
 - Data analysis agents
 - Verification agents



Manus.im Demo

Design for me an AI engineer's course



Okay, I can help you design a course for an AI engineer. I will create a plan to outline the course content and structure. This will involve researching the typical skills and knowledge required for this role and organizing them into modules or sections. I'll let you know once the plan is ready and as I make progress on the design.

Manus's computer

Manus is using Editor

Task progress 1 / 5

- Manus is working: Research AI engineering skills and industry requirements
1:01 Initializing the computer
- Design course structure and curriculum outline
- Develop detailed module content and learning materials
- Create practical projects and assessment framework
- Compile and deliver complete course design



I have completed the design of the AI Engineer's course. You can find the comprehensive course design document attached.



certified_ai_engineer_course....
PDF · 371.87 KB



1 of 40

40 pages report

Certified AI Engineer Course Design

Course Overview

This document outlines the comprehensive design for a "Certified AI Engineer" course,

AI Agents Demo

LangChain: Tool use, ReAct, Planner

AI_Agents.ipynb

```
import re
from langchain.agents import Tool, create_react_agent, AgentExecutor
from langchain.prompts import PromptTemplate
from langchain_openai import OpenAI
import textwrap
import json
```



<https://www.langchain.com/>

```
# --- Section 1: Tool Use ---
def add_numbers(a: int, b: int) -> int:
    """Add two integers and return the result."""
    return a + b

def is_even(number: int) -> bool:
    """Return True if number is even, else False."""
    return number % 2 == 0

def get_color_fact(color: str) -> str:
    """Return a fun fact about a color."""
    facts = {
        "blue": "Blue is the most commonly used favorite color worldwide.",
        "red": "Red is the first color a baby sees.",
        "yellow": "Yellow is the most luminous color."
    }
    return facts.get(color.lower(), "I don't have a specific fact for that color.")

print("--- Section 1: Tool Use ---")
print("Direct tool invocation: add_numbers(5, 3) =", add_numbers(5, 3))
print("Direct tool invocation: is_even(10) =", is_even(10))
print("Direct tool invocation: get_color_fact('blue') =", get_color_fact("blue"))
```

```
--- Section 1: Tool Use ---
Direct tool invocation: add_numbers(5, 3) = 8
Direct tool invocation: is_even(10) = True
Direct tool invocation: get_color_fact('blue') = Blue is the most commonly used
favorite color worldwide.
```

LangChain: Tool use, ReAct, Planner

```
# Wrap tools for agent
react_tools = [
    Tool(
        name="add_numbers",
        func=lambda arg: add_numbers(**(json.loads(arg) if isinstance(arg, str) else arg)),
        description="Add two numbers. Provide {\"a\": int, \"b\": int}"
    ),
    Tool(
        name="is_even",
        func=lambda arg: is_even(**(json.loads(arg) if isinstance(arg, str) else arg)),
        description="Check if a number is even. Provide {\"number\": int}"
    ),
    Tool(
        name="get_color_fact",
        func=lambda arg: get_color_fact(**(json.loads(arg) if isinstance(arg, str) else arg)),
        description="Return a fun fact about a color. Provide {\"color\": string}"
    ),
]
```

```
# --- Section 2: Custom ReAct Prompt Setup ---
tool_names = ", ".join(t.name for t in react_tools)
tools_md = "\n".join(f"***{t.name}***: {t.description}" for t in react_tools)
```

```
react_prompt = PromptTemplate(
    input_variables=["input", "agent_scratchpad"],
    partial_variables={"tools": tools_md, "tool_names": tool_names},
    template=textwrap.dedent("""
```

```
    You are a ReAct agent that can use the following tools:
    {tools}
```

```
    You must follow this format exactly:
```

```
    Question: the input question you must answer
```

```
    Thought: you should always think about what to do
```

```
    Action: the action to take, must be one of [{tool_names}]
```

```
    Action Input: the input to the action, as a JSON object
```

```
    Observation: the result of the action
```

```
    ... (this Thought/Action/Action Input/Observation cycle can repeat) ...
```

```
    Thought: I now know the final answer
```

```
    Final Answer: the answer to the original question
```

```
    Examples:
```

```
    Question: What is 5 plus 3?
```

```
    Thought: I need to use add_numbers.
```

```
    Action: add_numbers
```

```
    Action Input: {{ "a": 5, "b": 3 }}
```

```
    Observation: 8
```

```
    Thought: I now know the final answer.
```

```
    Final Answer: 8
```

```
    Question: Is 72 even?
```

```
    Thought: I should check with is_even.
```

```
    Action: is_even
```

```
    Action Input: {{ "number": 72 }}
```

```
    Observation: true
```

```
    Thought: I now know the final answer.
```

```
    Final Answer: Yes, 72 is even.
```

```
    Question: What's a fact about blue?
```

```
    Thought: I should use get_color_fact.
```

```
    Action: get_color_fact
```

```
    Action Input: {{ "color": "blue" }}
```

```
    Observation: Blue is the most popular color in the world.
```

```
    Thought: I now know the final answer.
```

```
    Final Answer: Blue is the most popular color in the world.
```

```
    Begin
```

```
    Question: {input}
```

```
    Thought:{agent_scratchpad}
```

```
    """)
```

```
)
```

LangChain: Tool use, ReAct, Planner

```
# Initialize your LLM
llm = OpenAI(temperature=0)

agent = create_react_agent(llm, react_tools, react_prompt)
agent_executor = AgentExecutor(agent=agent,
                               tools=react_tools,
                               verbose=True,
                               handle_parsing_errors=True
)

print("\n--- Section 2: ReAct Agent with Custom Prompt ---")
# Use invoke to call the chain (returns a dict)
# Provide both input and an empty scratchpad
print("Invoking agent via invoke()... (returns dict)")
test_input = "Is 72 an even number? Then tell me a fact about blue."
result = agent_executor.invoke({"input": test_input, "agent_scratchpad": ""})
# The agent's final answer is under the 'output' key
print("Agent Response:", result.get("output"))
```

```
--- Section 2: ReAct Agent with Custom Prompt ---
Invoking agent via invoke()... (returns dict)
```

```
> Entering new AgentExecutor chain...
I should first check if 72 is even using is_even, and then use get_color_fact
to get a fact about blue.
Action: is_even
Action Input: { "number": 72 }TrueAction: get_color_fact
Action Input: { "color": "blue" }Blue is the most commonly used favorite color
worldwide.I now know the final answer.
Final Answer: Yes, 72 is even and a fun fact about blue is that it is the most
commonly used favorite color worldwide.
```

```
> Finished chain.
Agent Response: Yes, 72 is even and a fun fact about blue is that it is the
most commonly used favorite color worldwide.
```


LangChain: Tool use, ReAct, Planner

```
# --- Section 3: Planner Execution ---
plan = [
    "1. Use add_numbers to find the sum of 100 and 200.",
    "2. Use is_even to check if 300 is an even number.",
    "3. Use get_color_fact to tell a fun fact about the color yellow."
]

print("\n--- Section 3: Planner Execution ---")
for step in plan:
    print(f"\nStep: {step}")
    name_match = re.search(r"Use (\w+)", step)
    if not name_match:
        print("Could not parse tool name.")
        continue
    tool_name = name_match.group(1)
    if tool_name == "add_numbers":
        nums = re.findall(r"(\d+)", step)[-2:] # Take last two numbers
        a, b = map(int, nums)
        result = add_numbers(a, b)
    elif tool_name == "is_even":
        num = int(re.search(r"(\d+)", step).group(1))
        result = is_even(num)
    elif tool_name == "get_color_fact":
        color = re.search(r"color (\w+)", step, re.IGNORECASE).group(1)
        result = get_color_fact(color)
    else:
        result = f"No handler for tool {tool_name}"
    print(f"Result: {result}")

print("\n--- Demo Complete ---")
```

--- Section 3: Planner Execution ---

Step: 1. Use add_numbers to find the sum of 100 and 200.
Result: 300

Step: 2. Use is_even to check if 300 is an even number.
Result: True

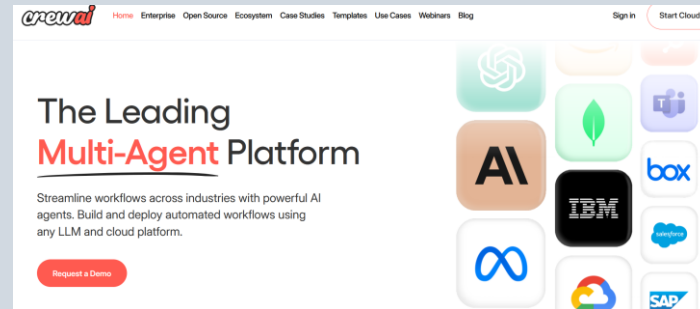
Step: 3. Use get_color_fact to tell a fun fact about the color yellow.
Result: Yellow is the most luminous color.

--- Demo Complete ---

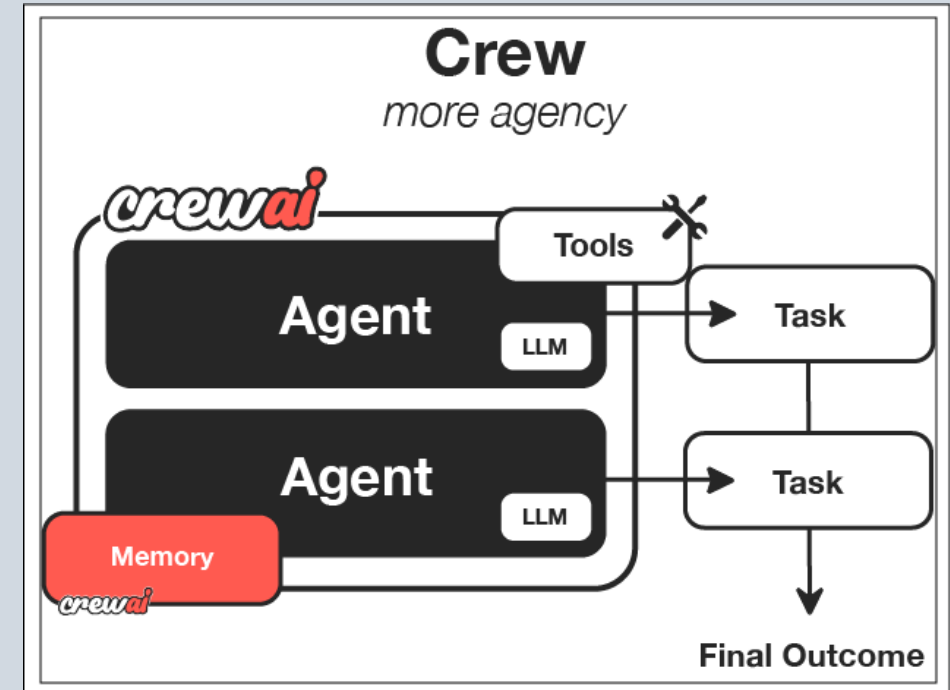
Crew AI



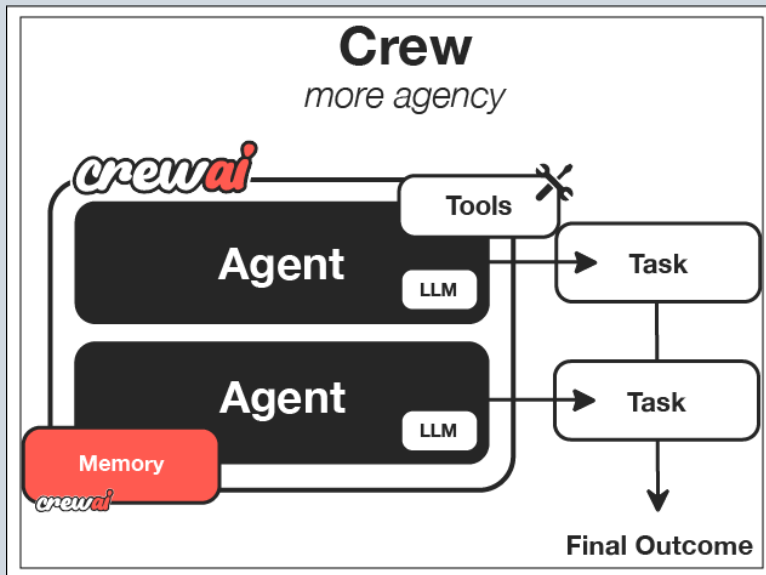
<https://www.crewai.com/>



- **CrewAI – Key Concepts**
 - **Agents** - Work on specialized tasks
 - **Tasks** defined to achieve the goal
 - **Process** - Collaboration between Agents
 - **Crew** organizes the overall operation to achieve final outcome



Crew AI



Component	Description	Key Features
Crew	The top-level organization	• Manages AI agent teams • Oversees workflows • Ensures collaboration • Delivers outcomes
AI Agents	Specialized team members	• Have specific roles (researcher, writer) • Use designated tools • Can delegate tasks • Make autonomous decisions
Process	Workflow management system	• Defines collaboration patterns • Controls task assignments • Manages interactions • Ensures efficient execution
Tasks	Individual assignments	• Have clear objectives • Use specific tools • Feed into larger process • Produce actionable results

Defining a Crew

1.  What's the Goal?
 - What do you want your crew of agents to accomplish?
 - Example: "Write a blog post about a new product based on customer reviews."
2.  Captain / Manager Agent
 - This is the "boss" agent that knows the big picture and gives orders.
 - Role (Job Title): Content Manager Agent
 - Goal: Plan, assign, and combine tasks into the final result
 - Backstory: Great at coordinating writing projects, knows who to assign what
 - Tools Used:
 - To-do planner (e.g. basic task list)
 - Chat interface (e.g. OpenAI Chat, Claude)
 - Simple decision logic (e.g. if...then rules)

Defining a Crew

3.  Crew Members (Expert Agents) What do you want your crew of agents to accomplish?
- Example: “Write a blog post about a new product based on customer reviews.”

Role	Goal	Backstory	Tools
Review Collector	Collect reviews from websites	Web-savvy, knows where people post reviews	Browser + Copy/Paste logic
Summary Writer	Turn messy reviews into a short summary	Strong writer, good at finding key points	OpenAI / Claude
SEO Helper	Suggest better blog title and keywords	Knows how Google search works	Simple keyword suggestion API
Proofreader	Fix grammar and clarity	Loves editing text	Grammarly API / GPT
Formatter	Turn final text into HTML or Markdown	Detail-focused, knows blog format	Markdown templates, Notepad

Defining a Crew

4. Common Tools & Resources

Purpose	Tool Examples
Text Generation	ChatGPT, Claude
Data Collection	Google Search, CSV export, scraping
Summarization	GPT Prompt: “Summarize this”
Formatting	Markdown, Google Docs
Collaboration	Google Sheets, Notion

CrewAI: Role-based collaboration.

```
from crewai import Agent, Task, Crew, Process
from crewai_tools import SerperDevTool, ScrapeWebsiteTool
from langchain_openai import ChatOpenAI
import os
```

```
# Define Agents
researcher = Agent(
    role="Market Research Analyst",
    goal="Research AI productivity tools",
    backstory="You specialize in market analysis.",
    tools=[search_tool, scrape_tool],
    llm=llm)

strategist = Agent(
    role="Content Strategist",
    goal="Develop launch content strategy",
    backstory="You craft strategies based on research.",
    llm=llm)

writer = Agent(
    role="Content Writer",
    goal="Write blog/social/email content",
    backstory="You write clear, engaging content.",
    llm=llm)

editor = Agent(
    role="Editor",
    goal="Edit and polish content",
    backstory="You ensure content quality and consistency.",
    llm=llm)
```

```
llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0.1)
search_tool = SerperDevTool()
scrape_tool = ScrapeWebsiteTool()
```

```
# Define Tasks
research = Task(
    description="Research top AI productivity tools, market size, trends, and competitors",
    agent=researcher,
    expected_output="Market report")

strategy = Task(
    description="Create content strategy based on research",
    agent=strategist,
    expected_output="Content strategy doc",
    context=[research])

writing = Task(
    description="Write blog, posts, email, and product description",
    agent=writer,
    expected_output="Content package",
    context=[strategy])

editing = Task(
    description="Edit content for consistency and quality",
    agent=editor,
    expected_output="Final edited package",
    context=[writing])
```

CrewAI: Role-based collaboration.

```
# Run Crew
crew = Crew(agents=[researcher, strategist, writer, editor], tasks=[research,
strategy, writing, editing], process=Process.sequential, verbose=True)

if __name__ == "__main__":
    if not os.getenv("OPENAI_API_KEY"):
        print("Set OPENAI_API_KEY")
        exit(1)
    result = crew.kickoff()
    print("✅ Final Output:\n", result)
```

```
# Agent: Market Research Analyst
## Task: Research top AI productivity tools, market size, trends, and competitors
```

```
# Agent: Market Research Analyst
## Thought: I need to start by researching the top AI productivity tools, market size, trends, and competitors to compile a comprehensive market report.
## Using tool: Search the internet with Serper
## Tool Input:
"{\"search_query\": \"top AI productivity tools 2021\"}"
## Tool Output:
{'searchParameters': {'q': 'top AI productivity tools 2021', 'type': 'search', 'num': 10, 'engine': 'google'}, 'organic': [{'title': 'The best AI producti
```

```
# Agent: Market Research Analyst
## Thought: Action: Read website content
## Using tool: Read website content
## Tool Input:
"{\"website_url\": \"https://zapier.com/blog/best-ai-productivity-tools/\"}"
## Tool Output:
The best AI productivity tools in 2025 | Zapier Skip to content Products Zapier Automation Platform No-code
```

Crew Execution Started

```
Crew Execution Started
Name: crew
ID: dd1dfdde-9e05-4429-98ff-1296502681a9
```

```
# Agent: Market Research Analyst
## Thought: Thought: I have gathered information on the top AI productivity tools from the Zapier blog.
## Using tool: Read website content
## Tool Input:
"{\"website_url\": \"https://zapier.com/blog/best-ai-productivity-tools/\"}"
## Tool Output:
I tried reusing the same input, I must stop using this action input. I'll try something else instead.

You ONLY have access to the following tools, and should NEVER make up tools that are not listed here:

Tool Name: Search the internet with Serper
Tool Arguments: {'search_query': {'description': 'Mandatory search query you want to use to search the internet', 'type': 'str'}}
Tool Description: A tool that can be used to search the internet with a search_query. Supports different search types:
Tool Name: Read website content
Tool Arguments: {'website_url': {'description': 'Mandatory website url to read the file', 'type': 'str'}}
Tool Description: A tool that can be used to read a website content.
```


CrewAI: Role-based collaboration.

```
# Run Crew
crew = Crew(agents=[researcher, strategist, writer, editor], tasks=[research,
strategy, writing, editing], process=Process.sequential, verbose=True)

if __name__ == "__main__":
    if not os.getenv("OPENAI_API_KEY"):
        print("Set OPENAI_API_KEY")
        exit(1)
    result = crew.kickoff()
    print("✅ Final Output:\n", result)
```

```
Crew: crew
└─ Task: f5e60cc5-d833-4703-bd84-6709c62710ae
   Assigned to: Market Research Analyst
   Status: ✅ Completed
   └─ Used Search the internet with Serper (1)
      └─ Used Read website content (1)
         └─ Used Search the internet with Serper (2)
            Task Completion (2)

Task Completed
Name: f5e60cc5-d833-4703-bd84-6709c62710ae
Agent: Market Research Analyst
```

✅ Final Output:
Title: Top AI Productivity Tools in 2025

Introduction:

In 2025, the landscape of AI productivity tools has evolved significantly, offering a wide range of solutions to enhance efficiency and streamline workflows. This comprehensive list highlights the top AI productivity tools that professionals can leverage to boost their productivity and achieve their goals.

List of Top AI Productivity Tools in 2025:

1. **ChatGPT**: A powerful AI tool for generating human-like text responses and assisting in various writing tasks.
2. **Claude**: An AI assistant designed to help professionals manage their ...

Conclusion:

The AI productivity tools listed above represent a diverse array of solutions that cater to various needs and preferences. By incorporating these tools into their workflows, professionals can optimize their productivity, streamline tasks, and achieve greater efficiency in their daily operations. Stay ahead of the curve in 2025 with these top AI productivity tools.

Planning a Trip with Crew AI

- Which agents do we need?
 - City Selection Agent
 - Local Expert
 - Travel Concierge
- Which tasks will they need to carry out?
 - Identify locations tasks
 - Gather information tasks
 - Plan trip tasks
- Which tools will they need?
 - Google search tool
 - Internet scraping tool
 - Calculator tools

[trip_agent_example_notebook.ipynb](#)

Defining the Agents

```
def city_selection_agent(self):
    return Agent(
        role='City Selection Expert',
        goal='Select the best city based on weather, season, and prices',
        backstory='An expert in analyzing travel data to pick ideal destinations',
        tools=[
            # call the plain function...
            search_internet,
            # ...and the staticmethod on BrowserTools
            BrowserTools.scrape_and_summarize_website,
        ],
        verbose=True)
```

```
def city_selection_agent(self):
    return Agent(
        role='City Selection Expert',
        goal='Select the best city based on weather, season, and prices',
        backstory='An expert in analyzing travel data to pick ideal destinations',
        tools=[
            # call the plain function...
            search_internet,
            # ...and the staticmethod on BrowserTools
            BrowserTools.scrape_and_summarize_website,
        ],
        verbose=True)
```

```
def travel_concierge(self):
    return Agent(
        role='Amazing Travel Concierge',
        goal=(
            "Create the most amazing travel itineraries with "
            "budget and "
            "packing suggestions for the city"
        ),
        backstory=(
            "Specialist in travel planning and logistics with "
            "decades of experience"
        ),
        tools=[
            search_internet,
            BrowserTools.scrape_and_summarize_website,
            # CalculatorTools.define a staticmethod, so call it here:
            CalculatorTools.calculate,
        ],
        verbose=True)
```

Defining the Tasks

```
def identify_task(self, agent, origin, cities, interests, range):
    return Task(
        description=dedent(f"""
            Analyze and select the best city for the trip based
            on specific criteria such as weather patterns, seasonal
            events, and travel costs. This task involves comparing
            multiple cities, considering factors like current weather
            conditions, upcoming cultural or seasonal events, and
            overall travel expenses.

            Your final answer must be a detailed
            report on the chosen city, and everything you found out
            about it, including the actual flight costs, weather
            forecast and attractions.
            {self.__tip_section()}

            Traveling from: {origin}
            City Options: {cities}
            Trip Date: {range}
            Traveler Interests: {interests}
            """),
        agent=agent,
        expected_output="Detailed report on the chosen city including flight
        costs, weather forecast, and attractions")
```

```
def gather_task(self, agent, origin, interests, range):
    return Task(
        description=dedent(f"""
            As a local expert on this city you must compile an
            in-depth guide for someone traveling there and wanting
            to have THE BEST trip ever!
            Gather information about key attractions, local customs,
            special events, and daily activity recommendations.
            Find the best spots to go to, the kind of place only a
            local would know.
            This guide should provide a thorough overview of what
            the city has to offer, including hidden gems, cultural
            hotspots, must-visit landmarks, weather forecasts, and
            high level costs.
            The final answer must be a comprehensive city guide,
            rich in cultural insights and practical tips,
            tailored to enhance the travel experience.
            {self.__tip_section()}
            Trip Date: {range}
            Traveling from: {origin}
            Traveler Interests: {interests}
            """),
        agent=agent,
        expected_output="Comprehensive city guide including hidden
        gems, cultural hotspots, and practical travel tips")
```

Defining the Tasks

```
def plan_task(self, agent, origin, interests, range):
    return Task(
        description=dedent(f"""
            Expand this guide into a full 7-day travel
            itinerary with detailed per-day plans, including
            weather forecasts, places to eat, packing suggestions,
            and a budget breakdown.
            You MUST suggest actual places to visit, actual hotels
            to stay and actual restaurants to go to.
            This itinerary should cover all aspects of the trip,
            from arrival to departure, integrating the city guide
            information with practical travel logistics.
            Your final answer MUST be a complete expanded travel plan,
            formatted as markdown, encompassing a daily schedule,
            anticipated weather conditions, recommended clothing and
            items to pack, and a detailed budget, ensuring THE BEST
            TRIP EVER. Be specific and give it a reason why you picked
            each place, what makes them special! {self.__tip_section()}
            Trip Date: {range}
            Traveling from: {origin}
            Traveler Interests: {interests}
            """),
        agent=agent,
        expected_output="Complete expanded travel plan with daily schedule,
            weather conditions, packing suggestions, and budget breakdown")
```

```
def __tip_section(self):
    return "If you do your BEST WORK, I'll tip you $100!"
```

Defining the Tools

```
def search_internet(query: str) -> str:
    """Search the internet about a given topic and return the top 4 results."""
    if not query:
        raise ValueError("search_internet requires a query string")
    payload = json.dumps({"q": query})
    headers = {
        "X-API-KEY": os.environ["SERPER_API_KEY"],
        "content-type": "application/json"
    }
    resp = requests.post("https://google.serper.dev/search", headers=headers,
data=payload)
    data = resp.json().get("organic", [])
    if not data:
        return "No results found (or check your SERPER_API_KEY)."
    snippets = []
    for r in data[:4]:
        snippets.append(
            f>Title: {r.get('title')}\n"
            f"Link: {r.get('link')}\n"
            f"Snippet: {r.get('snippet')}\n"
            "-" * 20
        )
    return "\n".join(snippets)
```

```
class CalculatorTools:
    @staticmethod
    @tool("Make a calculation")
    def calculate(operation):
        """Useful to perform any mathematical calculations,
        like sum, minus, multiplication, division, etc.
        The input to this tool should be a mathematical
        expression, a couple examples are `200*7` or `5000/2*10`
        """
        try:
            return eval(operation)
        except SyntaxError:
            return "Error: Invalid syntax in mathematical expression"
```

Defining the Tools

```
def scrape_and_summarize_website(website):
    """Useful to scrape and summarize a website content"""
    url =
    f"https://chrome.browserless.io/content?token={os.environ['BROWSERLESS_API_KEY']}"
    payload = json.dumps({"url": website})
    headers = {'cache-control': 'no-cache', 'content-type': 'application/json'}
    response = requests.request("POST", url, headers=headers, data=payload)
    elements = partition_html(text=response.text)
    content = "\n\n".join([str(el) for el in elements])
    content = [content[i:i + 8000] for i in range(0, len(content), 8000)]
    summaries = []
```

```
for chunk in content:
    agent = Agent(
        role='Principal Researcher',
        goal=
        'Do amazing researches and summaries based on the content you
        are working with',
        backstory=
        "You're a Principal Researcher at a big company and you need
        to do a research about a given topic.",
        allow_delegation=False)
    task = Task(
        agent=agent,
        description=
        f'Analyze and summarize the content bellow, make sure to
        include the most relevant information in the summary, return
        only the summary nothing else.\n\nCONTENT\n-----\n{chunk}'
    )
    summary = task.execute()
    summaries.append(summary)
return "\n\n".join(summaries)
```

Kickoff the Crew

```
def run(self):
    agents = TripAgents()
    tasks = TripTasks()

    city_selector_agent = agents.city_selection_agent()
    local_expert_agent = agents.local_expert()
    travel_concierge_agent = agents.travel_concierge()

    identify_task = tasks.identify_task(
        city_selector_agent, self.origin, self.cities, self.interests, self.date_range)
    gather_task = tasks.gather_task(
        local_expert_agent, self.origin, self.interests, self.date_range)
    plan_task = tasks.plan_task(
        travel_concierge_agent, self.origin, self.interests, self.date_range)

    crew = Crew(
        agents=[city_selector_agent, local_expert_agent, travel_concierge_agent],
        tasks=[identify_task, gather_task, plan_task],
        verbose=True)

    result = crew.kickoff()
    return result
```

```
if __name__ == "__main__":
    print("## Welcome to Trip Planner Crew")
    print('-----')
    location = input(
        dedent("""
            From where will you be traveling from? """))
    cities = input(
        dedent("""
            What are the cities options you are interested in visiting? """))
    date_range = input(
        dedent("""
            What is the date range you are interested in traveling? """))
    interests = input(
        dedent("""
            What are some of your high level interests and hobbies? """))

    trip_crew = TripCrew(location, cities, date_range, interests)
    result = trip_crew.run()
    print("\n\n#####")
    print("## Here is you Trip Plan")
    print("#####\n")
    print(result)
```


Get the Trip Plan

Welcome to Trip Planner Crew

From where will you be traveling from?

Jerusalem, Israel

What are the cities options you are interested in visiting?

New York, Boston

What is the date range you are interested in traveling?

1-31 August 2025

What are some of your high level interests and hobbies?

Hiking, sightseeing

#####

Here is you Trip Plan

#####

7-Day Travel Itinerary for New York City (August 2025)

Overview

- **Destination**: New York City
- **Travel Dates**: August 1 - August 7, 2025
- **Weather**: Expect temperatures from 24°C to 31°C (77°F to 88°F) with sunny days and occasional rain.
- **Packing Suggestions**: Lightweight clothing, a light jacket, comfortable shoes for walking, a raincoat or umbrella, and a reusable water bottle.

Accommodation

Hotel Suggestion:

Casablanca Hotel

- Located near Times Square with excellent amenities.
- Average price: \$300 per night.

Itinerary

Day 1: Arrival in New York City

- **Morning**: Arrive at JFK/LaGuardia and check into the Casablanca Hotel.
- **Afternoon**: Lunch at **Chelsea Market** for diverse food options.
- **Evening**: Explore **The High Line** for its elevated walking path and views (free entry).
- **Dinner**: Eat at **RH Rooftop Restaurant** with a stunning skyline view.
- **Budget**: \$75 (meals) + \$300 (hotel) = \$375

...

How CrewAI Shares Context Between Agents?

- Automatic Context Sharing in Crew.kickoff()
 - In CrewAI, when using Crew(...).kickoff(), previous task outputs are automatically appended to the prompt of subsequent agents.
 - This creates the illusion that agents "remember" each other's work — even though you didn't explicitly pass values.
 - It works like a shared reasoning log or conversation history.
- Example Behind the Scenes:
 - [Task 1 Output] "Barcelona was selected as the ideal city."
 - [Task 2 Prompt] "As a local expert on this city, write a guide..."
- LLMs can infer what "this city" refers to — because the previous output is part of the system prompt.

Comparing with LangChain Manual Prompt Chaining

[trip_agent_example_notebook_langchain.ipynb](#)

- LangChain Requires Explicit Prompt Management
 - In LangChain, you must manually capture and forward outputs between tasks.
 - There's no shared context — you control every step of the pipeline.
- Example:
 - `city_summary = llm.invoke(task1.description)`
 - `guide_prompt = f"Use this:\n{city_summary}\nWrite a travel guide..."`
 - `guide_summary = llm.invoke(guide_prompt)`
- CrewAI makes agent collaboration simple. LangChain gives you fine-grained control.

Feature	CrewAI kickoff()	LangChain .invoke()
Context sharing	✓ Implicit	✗ Manual
Flexibility	✗ Less control	✓ Full control
Effort required	✓ Minimal	⚠ Manual chaining

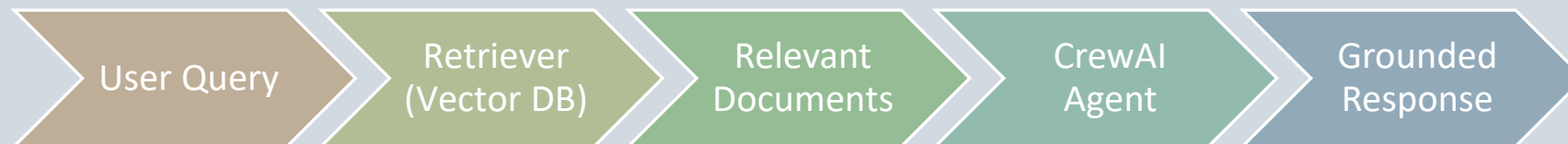
Your Turn

- Design an AI agent that will:
 - Parse the user's requirements (seating, cargo, budget, down payment, travel style)
 - Identify 5–7 car models available in Israel matching those needs
 - Compare each model on fuel efficiency, cargo capacity, total cost of ownership, and dealer availability
 - Generate a detailed, step-by-step purchase & negotiation plan tailored to the Israeli market
- Run the agent:
 - I want a car with seating for 5, ample cargo space for 2 large suitcases + strollers, good fuel economy for beach trips, within a total budget of ₪200000 and an initial down payment of ₪40000.
- Which agents do we need?
 - `model_selection_agent()`
 - `comparison_agent()`
 - `purchase_planner_agent()`
- Which tasks will they need to carry out?
 - `identify_models_task(agent, budget_ils, seating, cargo_needs, interests)`
 - `compare_models_task(agent, shortlist)`
 - `purchase_plan_task(agent, chosen_model, budget, down_payment)`
- Which tools will they need?
 - Google search tool
 - Internet scrapping tool
 - Calculator tools

CrewAI + RAG

[crewai_and_rag.ipynb](#)

- Grounding Agent Responses with RAG in CrewAI
- connect a CrewAI agent to a retriever that brings in relevant documents to support generation.
- This allows the agent to:
 - Answer questions with grounded evidence
 - Avoid hallucinations
 - Adapt responses based on real-time context
- This approach combines the structure of agent workflows with the depth of real-time, document-grounded knowledge.



Combine CrewAI and RAG

crewai_and_rag.ipynb

```
# ===== RAG: build/load =====
def load_pdf_chunks(pdf_path: str):
    pages = PyMuPDFLoader(pdf_path).load()
    splitter = RecursiveCharacterTextSplitter(chunk_size=700, chunk_overlap=100)
    chunks = splitter.split_documents(pages)
    # add stable metadata keys
    for i, c in enumerate(chunks):
        c.metadata.setdefault("source", pdf_path)
        c.metadata["chunk_id"] = i
    return chunks

def build_or_load_vstore() -> Chroma:
    os.makedirs(PERSIST_DIR, exist_ok=True)
    emb = HuggingFaceEmbeddings(model_name=EMBED_MODEL)
    # Reuse existing persisted collection if present
    if any(os.scandir(PERSIST_DIR)):
        return Chroma(collection_name="crew_rag_demo",
                       embedding_function=emb,
                       persist_directory=PERSIST_DIR)
    # Fresh build from the PDF
    docs = load_pdf_chunks(SOURCE_PDF)
    return Chroma.from_documents(docs, emb,
                                collection_name="crew_rag_demo",
                                persist_directory=PERSIST_DIR)

VSTORE = build_or_load_vstore()
```

Combine CrewAI and RAG

```
# ===== Agent #1: Research =====
research_agent = Agent(
    role="Evidence Research Agent",
    goal="Answer using the private PDF KB (RAG) first. Cite [KB-#].",
    backstory="You ground answers in the internal PDF knowledge base.",
    tools=[rag_search],
    llm=llm,
    verbose=True,
    allow_delegation=False,
    max_iter=4,
)

task_research = Task(
    description=(
        "Use the RAG tool to gather evidence and produce 3 concise bullets.\n"
        "Include a 'Citations' section listing the [KB-#] you used.\n"
        f"User question: {QUESTION}"
    ),
    expected_output="Markdown with 3 bullets and a 'Citations' section referencing [KB-#].",
    agent=research_agent,
)
```

Image Agent Demo

[Agents_image_demo.ipynb](#)

- Showcase how an AI agent can analyze **visual inputs** and generate insights.
 - Extracting text and metadata from images (OCR + vision models).
 - Providing descriptions, classifications, and summaries.
 - Combining tool outputs with reasoning (e.g., describe an image and then suggest context or use cases).
- **Key Benefits:**
 - Enables document digitization, medical imaging insights, and multimedia knowledge bases.
 - Demonstrates how agents bridge raw **visual data** → **structured knowledge**.

Audio Agent Demo (Speech-to-Insights)

[Agents_audio_demo.ipynb](#)

- Showcase how an AI agent can transform **raw audio** into structured, actionable insights.
 - Get audio (download a sample if missing).
 - Transcribe to text using local Whisper or OpenAI API.
 - Summarize the transcript with a lightweight LLM prompt.
 - Run an agent workflow (CrewAI) to automate: download → transcribe → analyze → save report..
- **Key Benefits:**
 - Turns spoken content into searchable text and concise reports.
 - Increases accessibility (meeting minutes, call transcripts, podcasts).
 - Supports compliance, customer support, and research by converting **voice** → **structured knowledge**.

YouTube Video Agent Demo

[Agents_youtube_video_demo.ipynb](#)

- Showcase how an AI agent can process **unstructured video content** and transform it into actionable knowledge.
 - Fetching YouTube captions or transcripts.
 - Summarizing content into concise highlights or Q&A.
 - Using external APIs + reasoning for **media intelligence**.
- **Key Benefits:**
 - Enables rapid knowledge acquisition from long-form videos.
 - Supports education, research, compliance monitoring, and enterprise training.
 - Transforms passive video consumption into **structured, searchable insights**.

Using uv with CrewAI

- What is uv?
 - Next-gen Python package & environment manager(by Astral — creators of Ruff & Rye)
 - Unifies the ecosystem:
 - pip → installs
 - virtualenv/venv → environments
 - pip-tools → locking & reproducibility
 - Key advantages:
 - ⚡ Blazing-fast installs (Rust-based, parallelized)
 - 📦 Deterministic builds via uv.lock
 - 🛠 Simple commands: uv add, uv run, uvx
 - 🌐 Cross-platform consistency (same lock on Mac, Linux, Windows)
- uv simplifies the messy Python toolchain into one fast, reliable workflow. It makes setting up environments, adding dependencies, running scripts, and locking versions effortless.
- For CrewAI and multi-agent systems, this means less time fighting environments and more time iterating on agents.

<https://docs.astral.sh/uv/>

Using uv with CrewAI

crewai_uv project

- Step 1: Install uv (fast Python package manager)
 - uv handles both its own install and tool installation in one go

```
# Install the uv package manager:  
# macOS / Linux  
curl -LsSf https://astral.sh/uv/install.sh | sh  
  
# Windows (PowerShell)  
powershell -ExecutionPolicy ByPass -c "irm  
https://astral.sh/uv/install.ps1 | iex"  
  
# Then install the CrewAI CLI:  
uv tool install crewai # installs the CLI  
uv tool list           # should show something like: crewai v0.xxx
```

<https://docs.crewai.com/en/installation>

Using uv with CrewAI

- Step 2: Scaffold a new CrewAI project

- This generates:

- .env (for keys), pyproject.toml,
 - src/.../config/agents.yaml & tasks.yaml,
 - src/.../crew.py and main.py CrewAI+2CrewAI+2GitHub+1.

```
crewai create crew my-project  
cd my-first-crew
```

- Customize Your Project

- Your project will contain these essential files:

File	Purpose
agents.yaml	Define your AI agents and their roles
tasks.yaml	Set up agent tasks and workflows
.env	Store API keys and environment variables
main.py	Project entry point and execution flow
crew.py	Crew orchestration and coordination
tools/	Directory for custom agent tools
knowledge/	Directory for knowledge base

```
my_project/  
├── .gitignore  
├── knowledge/  
├── pyproject.toml  
├── README.md  
├── .env  
└── src/  
    ├── my_project/  
    │   ├── __init__.py  
    │   ├── main.py  
    │   ├── crew.py  
    │   └── tools/  
    │       └── custom_tool.py  
    └── config/  
        ├── __init__.py  
        └── config/  
            ├── agents.yaml  
            └── tasks.yaml
```

- Start by editing agents.yaml and tasks.yaml to define your crew's behavior.
 - Keep sensitive information like API keys in .env.

Using uv with CrewAI

- Step 3: Install dependencies & lock the environment

- Before you run your crew, make sure to run: *crewai install*
- Behind the scenes, this runs *uv sync* or *uv install* to set up and lock dependencies reliably
- If you need to install additional packages, use:
 - *uv add <package-name>*

```
crewai install
```

- Step 4. Run your agent crew

- *crewai run*
- This executes your crew using the locked environment, launching your agents and tasks as defined in the YAML and Python scaffolding

```
crewai run
```

Crewai uv example: Define your Agents and Taks

```
▼ crewai_uv
  > .venv
  > knowledge
  ▼ src
    ▼ crewai_uv
      ▼ config
        ! agents.yaml
        ≡ desktop.ini
        ! tasks.yaml
      ▼ tools
        🛠 __init__.py
        🛠 custom_tool.py
        ≡ desktop.ini
        🛠 __init__.py
        🛠 crew.py
        ≡ desktop.ini
        🛠 main.py
```

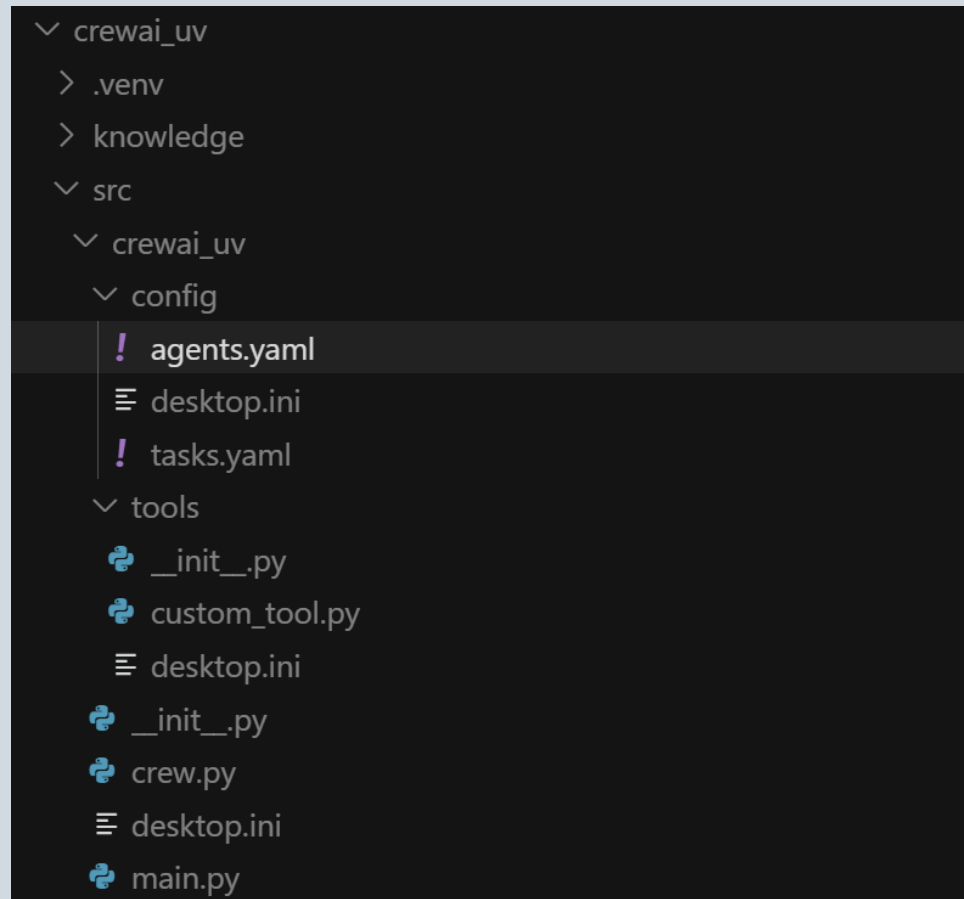
```
@CrewBase
class CrewaiUv():
    """CrewaiUv crew"""

    agents: List[BaseAgent]
    tasks: List[Task]
    @agent
    def researcher(self) -> Agent:
        return Agent(
            config=self.agents_config['researcher'],
            verbose=True)

    @agent
    def reporting_analyst(self) -> Agent:
        return Agent(
            config=self.agents_config['reporting_analyst'],
            verbose=True )

    @task
    def research_task(self) -> Task:
        return Task(
            config=self.tasks_config['research_task'] )
```

Crewai uv example: agents.yaml



```
researcher:
  role: >
    {topic} Senior Data Researcher
  goal: >
    Uncover cutting-edge developments in {topic}
  backstory: >
    You're a seasoned researcher with a knack for uncovering the latest
    developments in {topic}. Known for your ability to find the most relevant
    information and present it in a clear and concise manner.

reporting_analyst:
  role: >
    {topic} Reporting Analyst
  goal: >
    Create detailed reports based on {topic} data analysis and research
    findings
  backstory: >
    You're a meticulous analyst with a keen eye for detail. You're known for
    your ability to turn complex data into clear and concise reports, making
    it easy for others to understand and act on the information you provide.
```


Crewai uv example: tasks.yaml

```

└─ crewai_uv
  └─ .venv
  └─ knowledge
  └─ src
    └─ crewai_uv
      └─ config
        └─ ! agents.yaml
        └─ desktop.ini
        └─ ! tasks.yaml
      └─ tools
        └─ _init__.py
        └─ custom_tool.py
        └─ desktop.ini
        └─ _init__.py
        └─ crew.py
        └─ desktop.ini
        └─ main.py
```

```
research_task:
  description: >
    Conduct a thorough research about {topic}
    Make sure you find any interesting and relevant information given
    the current year is {current_year}.
  expected_output: >
    A list with 10 bullet points of the most relevant information about
    {topic}
  agent: researcher

reporting_task:
  description: >
    Review the context you got and expand each topic into a full section for
    a report.
    Make sure the report is detailed and contains any and all relevant
    information.
  expected_output: >
    A fully fledged report with the main topics, each with a full section of
    information.
    Formatted as markdown without ```
  agent: reporting_analyst
```

Crewai uv example: Create Custom tools

```
▼ crewai_uv
  > knowledge
  ▼ src
    ▼ crewai_uv
      ▼ config
        ! agents.yaml
        ≡ desktop.ini
        ! tasks.yaml
      ▼ tools
        📄 __init__.py
        📄 custom_tool.py
        ≡ desktop.ini
        📄 __init__.py
        📄 crew.py
        ≡ desktop.ini
        📄 main.py
```

```
from crewai.tools import BaseTool
from typing import Type
from pydantic import BaseModel, Field

class MyCustomToolInput(BaseModel):
    """Input schema for MyCustomTool."""
    argument: str = Field(..., description="Description of the argument.")

class MyCustomTool(BaseTool):
    name: str = "Name of my tool"
    description: str = (
        "Clear description for what this tool is useful for, your agent will need "
        "this information to use it."
    )
    args_schema: Type[BaseModel] = MyCustomToolInput

    def _run(self, argument: str) -> str:
        # Implementation goes here
        return "this is an example of a tool output, ignore it and move along."
```

Crewai uv example: Run the crew

```
✓ crewai_uv
  > knowledge
  ✓ src
    ✓ crewai_uv
      ✓ config
        ! agents.yaml
        ≡ desktop.ini
        ! tasks.yaml
      ✓ tools
        📄 __init__.py
        📄 custom_tool.py
        ≡ desktop.ini
        📄 __init__.py
        📄 crew.py
        ≡ desktop.ini
        📄 main.py
```

```
def run():
    """Run the crew."""
    inputs = {
        'topic': 'MCP framework for medical health management',
        'current_year': str(datetime.now().year)
    }

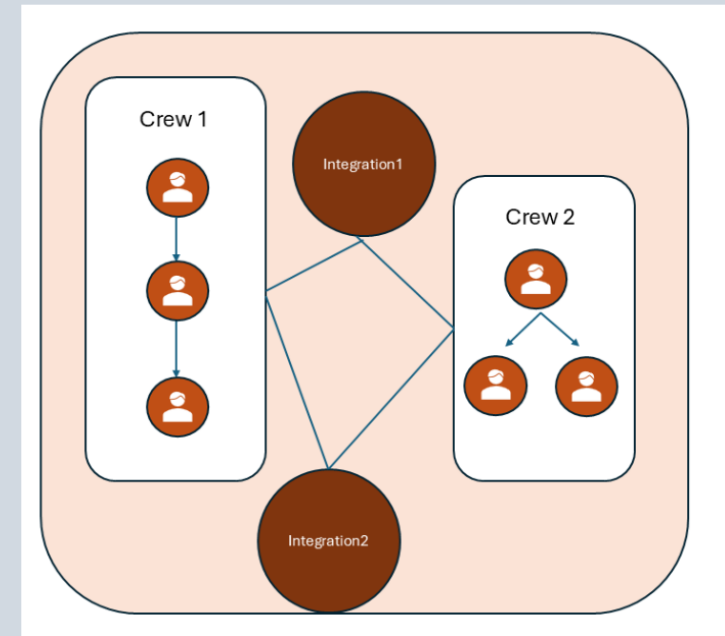
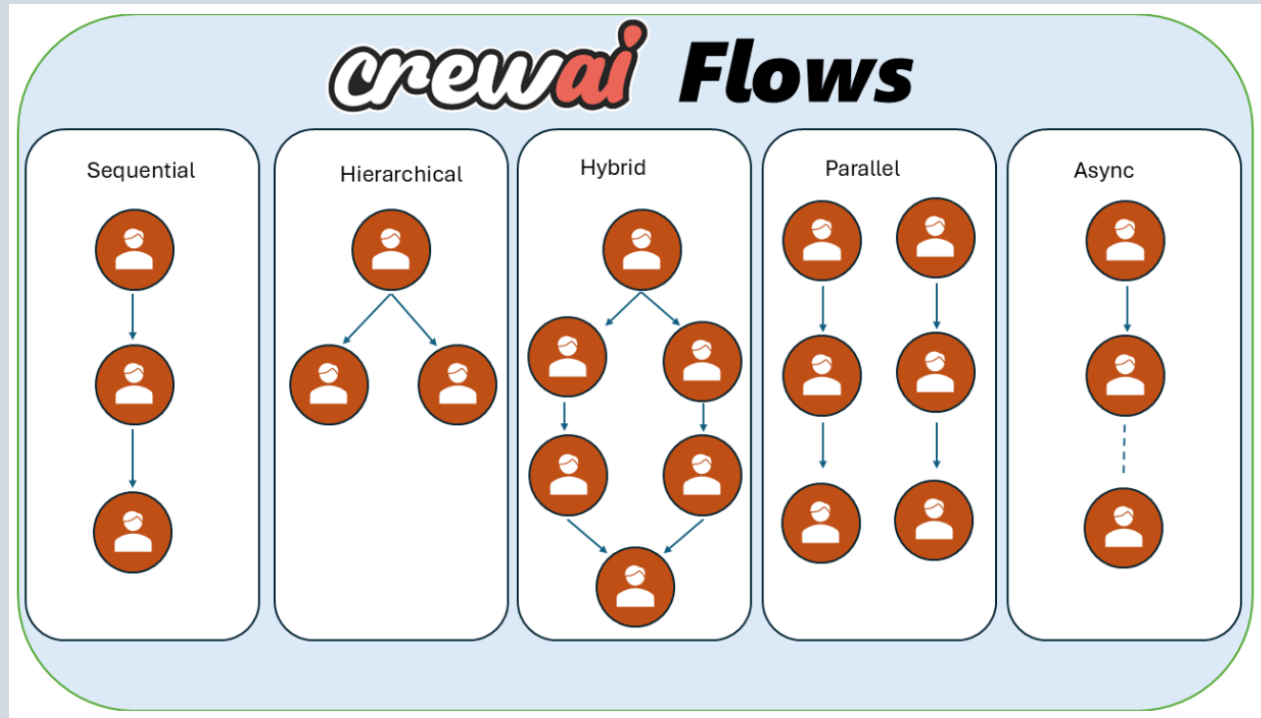
    try:
        CrewaiUv().crew().kickoff(inputs=inputs)
    except Exception as e:
        raise Exception(f"An error occurred while running the crew: {e}")

def train():
    """Train the crew for a given number of iterations."""
    inputs = {
        'topic': "AI LLMs",
        'current_year': str(datetime.now().year)
    }

    try:
        CrewaiUv().crew().train(n_iterations=int(sys.argv[1]), filename=sys.argv[2], inputs=inputs)

    except Exception as e:
        raise Exception(f"An error occurred while training the crew: {e}")
```

Crewai Flow



Crewai Flow

- **CrewAIFlows** are a structured, event-driven framework designed to **orchestrate** AI workflows.
- They allow you to coordinate multiple tasks, whether code-based, direct LLM calls, or collaborative agent Crews into a seamless, controllable pipeline.
- Flows elevate AI orchestration by offering:
 - **Modularity & orchestration**: Easily chain tasks and Crews into well-structured workflows
 - **Reactive design**: Workflows are responsive and can adapt based on earlier results or external inputs
 - **Visibility & control**: You can visualize the flow structure using `.plot()` or CLI commands to generate diagrams, helping understand and debug complex flows

Flows Components

- **Event-driven execution:**
 - Tasks trigger in response to specific events or outputs using decorators like `@start()`, `@listen()`, and `@router()`
- **State management:**
 - Flows maintain shared context through either unstructured (flexible dict-style) or structured (Pydanticmodel-based) state mechanisms
- **Conditional logic and routing:**
 - You can direct workflow paths dynamically using constructs like `or_`, `and_`, and `@router()` to handle branching logic based on state or task outcomes

Agentic Sales Pipeline – Use Case & Examples

sales_uv_demo project

- Automating the *early sales funnel* with AI agents that:
 - **Research** potential leads (company overview, tech stack, pain points)
 - **Qualify** them against ICP fit and intent signals
 - **Draft personalized outreach emails** with a clear CTA
 - **Summarize** into a concise sales report for stakeholders
- **Demo Examples (CrewAI modes):**
 - **Sequential Flow**
→ *Research* → *Qualification* → *Email* → *Report*
 - **Hierarchical Flow**
→ *Manager agent* orchestrates tasks, delegates, and validates outputs
 - **Hybrid Flow**
→ *Sequential pipeline* that spins up a **sub-crew** (deep-dive researchers) before returning to main flow
 - **Reactive Flow (@start/@listen)**
→ *Event-driven*: new topic triggers research → qualification → outreach → report automatically

Agentic Sales Pipeline – Use Case & Examples

sales_uv_demo project

- Top-level Sales Crew

```
@crew
def crew(self) -> Crew:
    """
    Choose process by env var:
    SEQUENTIAL: process=sequential
    HIERARCHICAL: process=hierarchical + manager (agent or llm)
    HYBRID: sequential top-level but includes hybrid_fanout_task before report
    """

    import logging
    mode = os.getenv("SALES_DEMO_MODE", "sequential").lower()
    tasks = [self.research_task(), self.qualification_task(), self.email_task(), self.report_task()]
```

```
return Crew(
    agents=[self.researcher(), self.reporter()],
    tasks=[
        Task(description="Fallback research", expected_output="Notes", agent=self.researcher()),
        Task(description="Fallback report", expected_output="Markdown", agent=self.reporter())
    ],
    process=Process.sequential,
    verbose=True
)
```


Agentic Sales Pipeline – Use Case & Examples

sales_uv_demo project

- Top-level Sales Crew

```
@CrewBase
class SalesCrew:
    """ Demonstrates sequential, hierarchical, and hybrid flows. """
    agents: List[BaseAgent]
    tasks: List[Task]

    # --- Agents (wired from YAML) ---
    @agent
    def manager(self) -> Agent:
        return Agent(config=self.agents_config['manager'], verbose=True)

    @agent
    def researcher(self) -> Agent:
        return Agent(config=self.agents_config['researcher'], verbose=True)

    @agent
    def qualifier(self) -> Agent:
        return Agent(config=self.agents_config['qualifier'], verbose=True)

    @agent
    def outreach(self) -> Agent:
        return Agent(config=self.agents_config['outreach'], verbose=True)

    @agent
    def reporter(self) -> Agent:
        return Agent(config=self.agents_config['reporter'], verbose=True)
```

```
@task
def qualification_task(self) -> Task:
    return Task(config=self.tasks_config['qualification_task'])

@task
def email_task(self) -> Task:
    return Task(config=self.tasks_config['email_task'])

@task
def report_task(self) -> Task:
    return Task(config=self.tasks_config['report_task'])

# --- Hybrid fan-out: hierarchical sub-crew ---
@task
def hybrid_fanout_task(self) -> Task:
    def _run(context: Dict[str, Any]) -> str:
        # Pass topic/leads to sub-crew
        sub_result = DeepDiveSubCrew().crew().kickoff(inputs=context)
        return f"[HYBRID SUB-CREW RESULT]\\n{sub_result}"

    return Task(
        description=(
            "Kick off a hierarchical sub-crew to perform focused deep dives per lead, "
            "then return a merged summary for the next task."
        ),
        expected_output="Merged deep-dive summary returned to the main pipeline.",
        agent=self.researcher(),
        function=_run    )
```

Agentic Sales Pipeline – Use Case & Examples

sales_uv_demo project

- Hierarchical sub-crew for "Hybrid"

```
@CrewBase
class DeepDiveSubCrew:
    agents: List[BaseAgent]
    tasks: List[Task]

    @agent
    def manager(self) -> Agent:
        return Agent(config=self.agents_config['manager'], verbose=True)

    @agent
    def specialist_research(self) -> Agent:
        # Reuse 'researcher' config or add a dedicated specialist in YAML
        cfg = self.agents_config.get('specialist_research', self.agents_config['researcher'])
        return Agent(config=cfg, verbose=True)

    @agent
    def specialist_risk(self) -> Agent:
        cfg = self.agents_config.get('specialist_risk', self.agents_config['qualifier'])
        return Agent(config=cfg, verbose=True)
```

```
@task
def dive_arch_task(self) -> Task:
    return Task(
        description=(
            "Deep dive on each lead focusing on architecture/stack and integration touchpoints. "
            "Return crisp bullets with sources."
        ),
        agent=self.specialist_research()
    )

@task
def dive_risk_task(self) -> Task:
    return Task(
        description=(
            "Deep dive on each lead focusing on risks, compliance, and likely blockers. "
            "Provide mitigation notes."
        ),
        agent=self.specialist_risk()
    )
```

Agentic Sales Pipeline – Use Case & Examples

sales_uv_demo project

- Hierarchical sub-crew for "Hybrid"

```
@crew
def crew(self) -> Crew:
    """
    Choose process by env var:
    SEQUENTIAL: process=sequential
    HIERARCHICAL: process=hierarchical + manager (agent or llm)
    HYBRID: sequential top-level but includes hybrid_fanout_task before report
    """
    import logging
    mode = os.getenv("SALES_DEMO_MODE", "sequential").lower()
    tasks = [self.research_task(), self.qualification_task(), self.email_task(), self.report_task()]

    if mode == "hybrid":
        tasks = [self.research_task(), self.hybrid_fanout_task(), self.qualification_task(), self.email_task(),
                self.report_task()]
```

```
if mode == "hierarchical":
    # Try manager_agent (newer CrewAI)
    mgr_agent = self.manager()
    try:
        return Crew(
            agents=[mgr_agent, self.researcher(), self.qualifier(), self.outreach(), self.reporter()],
            tasks=tasks,
            process=Process.hierarchical,
            manager_agent=mgr_agent,
            verbose=True
        )
    except Exception as e1:
        logging.warning("manager_agent path failed: %s", e1)
        # Fallback to manager_llm (older CrewAI)
```

...

Agentic Sales Pipeline – Use Case & Examples

sales_uv_demo project

- Reactive (@start / @listen) demo

```
class ReactiveSalesFlow:
    """
    Reactive flow that demonstrates @start and @listen.
    """

    def __init__(self, agents_config: Dict[str, Any]):
        self._researcher = Agent(config=agents_config['researcher'], verbose=True)
        self._qualifier = Agent(config=agents_config['qualifier'], verbose=True)
        self._outreach = Agent(config=agents_config['outreach'], verbose=True)
        self._reporter = Agent(config=agents_config['reporter'], verbose=True)

    @start()
    def new_topic(self, topic: str, leads: List[Dict[str, Any]] | None = None) -> Dict[str, Any]:
        return {"topic": topic, "leads": leads or []}
```

```
@listen(new_topic)
def do_research(self, payload: Dict[str, Any]) -> Dict[str, Any]:
    topic = payload["topic"]
    leads = payload["leads"]
    prompt = f"Research topic: {topic}. Leads: {json.dumps(leads)}"
    notes = self._researcher.execute(prompt)
    return {"topic": topic, "leads": leads, "research_notes": notes}

@listen(do_research)
def qualify(self, payload: Dict[str, Any]) -> Dict[str, Any]:
    notes = payload["research_notes"]
    leads = payload["leads"]
    prompt = f"Given notes: {notes}\nScore each lead (0-100) with rationale. Return JSON."
    scores = self._qualifier.execute(prompt)
    return {"**payload", "scores": scores}

@listen(qualify)
def write_emails(self, payload: Dict[str, Any]) -> Dict[str, Any]:
    leads = payload["leads"]
    notes = payload["research_notes"]
    prompt = f"Draft short emails for leads {json.dumps(leads)} using notes: {notes}"
    emails = self._outreach.execute(prompt)
    return {"**payload", "emails": emails}

@listen(write_emails)
def make_report(self, payload: Dict[str, Any]) -> str:
    scores = payload.get("scores", "")
    emails = payload.get("emails", "")
    prompt = f"Create a concise Markdown report. Scores: {scores}\nEmails: {emails}"
    return self._reporter.execute(prompt)
```

Enterprise Installation Options

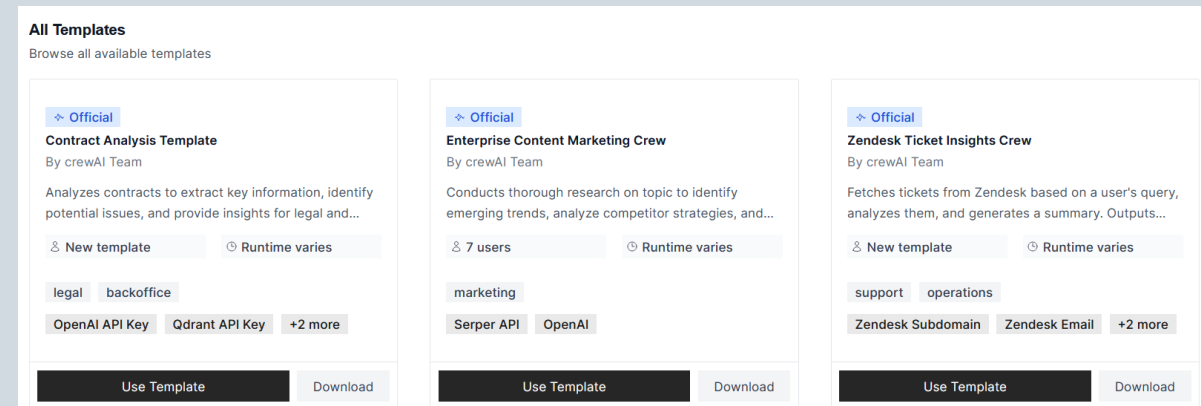
- For teams and organizations, CrewAI offers enterprise deployment options that eliminate setup complexity:

- **CrewAI Enterprise (SaaS)**

- Zero installation required - just sign up for free at app.crewai.com
- Automatic updates and maintenance
- Managed infrastructure and scaling
- Build Crews with no Code

- **CrewAI Factory (Self-hosted)**

- Containerized deployment for your infrastructure
- Supports any hyperscaler including on prem deployments
- Integration with your existing security systems



- For more details, see <https://www.crewai.com/enterprise>

Security & Monitoring

1. Authentication & Access Control

- JWT tokens for secure API access
- Role-based permissions (e.g., user, admin)

2. Safe Tool Execution

- Sandbox/Docker for code tools
- Timeout & memory limits

3. Input Protection

- Sanitize user inputs
- Prompt injection defense with guardrails

4. Logging & Audit Trails

- Log tool/LLM usage, inputs, outputs
- Store structured logs for traceability

5. Monitoring & Alerts

- Track latency, success rate, token usage
- Set alerts for anomalies or abuse

6. Rate Limiting

- Enforce per-user/IP request limits

Prompt Engineering for Agents

1. Define Clear Tool Instructions

- Explicit tool names, input/output formats
- Avoid generic names (e.g., “tool1” → “search_web”)
- Use consistent schemas ({ "a": int, "b": int })

2. Use ReAct or Plan-and-Execute Patterns

- Include structure in prompt: Thought, Action, Action Input, Observation, Final Answer

3. Encourage Step-by-Step Reasoning

- Add system prompt instructions:
 - “Think step by step before choosing a tool.”
- Improves accuracy and tool selection

4. Control Agent Behavior

- Use system-level guidance to:
 - Limit hallucinations
 - Avoid misuse of tools
 - Enforce reasoning flow

5. Test and Refine Prompts

- A/B test different phrasing
- Log token usage, output quality
- Tune for different models (GPT, Claude, Gemini)

6. Modular Prompt Design

- Separate: Tool descriptions, Example interactions, Task-specific instructions
- Easier reuse across agents

Evaluation & Pitfalls

Metric	Description
Task Success	Did the agent achieve the user's goal?
Tool Accuracy	Were the correct tools used with correct inputs?
Latency	Time from user input to final response
Token Efficiency	Total tokens used vs. output quality
User Satisfaction	Feedback from real users or test cases

Common Pitfalls to Watch For

- Prompt Injection & Tool Abuse
 - Users craft inputs that break tool logic or extract internal behavior.
 - Mitigation: input sanitization + strict schema validation.
- Infinite Reasoning Loops
 - Agents stuck in Thought → Action → Observation cycles.
 - Solution: max iterations or early exit conditions.
- Tool Misuse
 - Wrong tool selected, invalid input format, or broken output parsing.
 - Solution: better tool descriptions, validation, structured responses.
- Missing or Leaky Memory
 - Stateless agents forget previous steps or reuse stale context.
 - Fix: use memory modules (e.g., conversation buffer, vector store).
- Latency Bottlenecks
 - Especially in multi-agent or RAG workflows.
 - Solution: async tool calls, caching, parallelism where possible.

Real-World Use Cases

- Healthcare:
 - Summarize notes, suggest plans.
- DevOps:
 - Read logs, trigger alerts.
- Finance:
 - Review anomalies, summarize reports.
- Education:
 - Tutor + evaluator agent combo.

Further Resources

- **Documentation & Guides**

- LangChain Agents: <https://docs.langchain.com/docs/components/agents/>
- OpenAI Function Calling/Tool Use Documentation: <https://platform.openai.com/docs/guides/function-calling>
- Microsoft Research on AutoGen: <https://www.microsoft.com/en-us/research/project/autogen/>
- CrewAI Documentation: <https://docs.crewai.com/>
- Best Practices for Prompt Engineering with Agents <https://www.deeplearning.ai/short-courses/prompt-engineering-with-Llama-2/>

- **Repositories**

- GenAI Agents: https://github.com/NirDiamant/GenAI_Agents/tree/main
- LangServe: <https://github.com/langchain-ai/langserve>
- AutoGen: <https://github.com/microsoft/autogen>
- CrewAI: <https://github.com/joaoomdmoura/crewai>
- Crewai Flows Crash Course: <https://github.com/bhancockio/crewai-flows-crash-course/tree/main>

- **YouTube videos**

- Project Mariner: https://www.youtube.com/watch?v=kY_Zt5l-240
- Agent Overview: <https://www.youtube.com/watch?v=21-minutes-agent-overview>
- LangChain Agents with Tools: <https://www.youtube.com/watch?v=langchain-tools-video>
- LangChain Planning Agent Tutorial: <https://www.youtube.com/watch?v=langchain-planning-agent>
- AutoGen Multi-Agent Demo: <https://www.youtube.com/watch?v=autogen-multi-agent>
- LangServe Deployment Guide: <https://www.youtube.com/watch?v=langserve-deploy>
- General AI Agent Concepts: <https://www.youtube.com/watch?v=general-ai-agents-concepts>

Wrap-Up & Q&A

- **Key Takeaways:**

- Agents enable intelligent, autonomous workflows.
- Frameworks like LangChain, AutoGen, and CrewAI make agent development easier.
- MCP architecture ensures scalable deployment.
- Evaluate agents using logs, metrics, and user tests.

Wrap-Up & Q&A

- **Next Steps**
 - Clone example agents from LangChain.
 - Add memory + tools.
 - Try multi-agent planning in CrewAI.
 - Deploy with LangServe.
 - Integrate with your frontend (Angular or React).
 - Try to implement Agents and MCP in your own projects.

Upcoming Meetings

- Module 25: LLM Methodology & Ecosystem (16.09 – Frontal)
- Module 26: LLM as an instrument - Data cleansing - LLM ready (18.09 – Zoom)
- Module 27: Embedding Strategies, Retrieval Pipelines, & Vector Stores (25.09 – Zoom)
- Module 28: RAG Architecture & Variants (30.09 – Zoom)
- Module 29: AI Agents (16.10 – Zoom)
- **Module 30: MCP Architecture (21.10 – Zoom)**
- **Module 31: Meeting at AWS (23.10)**
- **Module 32: MCP Architecture Cont. (28.10 – Zoom)**
- **Project presentations, graduation ceremony (30.10)**

THANK YOU FOR LISTENING

ZVI.BENAMI@MAIL.HUJI.AC.IL

